דוח חקר פגיעות אבטחה: ניצול שרשור חולשות (XSS ל-CSRF) ויצירת תולעת רשת (Web Worm) אוטונומית בסביבה מרובת-הגנות (CORS, CSP, HttpOnly)

שם פרטי (שם משפחה)

ישראל שמש , עידו אגאי

שם בית הספר

עזריאלי

תיאור החולשה והרקע התאורטי

מהי החולשה

הפרויקט מציג הוכחת היתכנות (POC) מורכבת של מתקפה המבוססת על שרשור חולשות (Vulnerability Chaining). החולשה מורכבת משילוב של:

1. **Stored Cross-Site Scripting (XSS):** הזרקת קוד זדוני הנשמר במסד הנתונים ורץ בדפדפן הקורבן בעת קריאת הודעה.
2. **Cross-Site Request Forgery (CSRF) Bypass:** עקיפת מנגנון אסימוני האבטחה (Tokens) של השרת, המאפשרת לקוד הזדוני לפעול בשם הקורבן.

השילוב איפשר פיתוח של **תולעת רשת (Web Worm)** אוטונומית לחלוטין. ברגע שמשתמש צופה בהודעה הנגועה, התולעת משתלטת על הסשן שלו, עוקפת מנגנוני הגבלת קצב (Rate Limiter), מפיצה את עצמה לכלל אנשי הקשר שלו, ואף פותחת ערוץ שליטה ובקרה (C2) לדיווח חזרה לתוקף.

כיצד היא נוצרת (תרחיש ההדגמה)

כדי להדגים כיצד פגיעות כה קריטית מגיעה לסביבת ייצור (Production) המוגנת היטב, נבנה תרחיש מציאותי של פיתוח תחת לחץ:

באפליקציית הצ'אט, מנהל המוצר (Product Manager) דרש להוסיף פיצ'ר של "מדיה עשירה" (Rich Media) – היכולת לנגן סרטוני YouTube ישירות בתוך השיחה, ללא פתיחת חלונות חיצוניים. עקב דד-ליין דוחק, צוות הפיתוח בחר בפתרון מהיר:

במקום לבנות רכיב React ייעודי ובטוח, הם אפשרו הכנסת תגיות <iframe>. במקביל, כדי לתמוך בסקריפטים נלווים למדיה, הם הוסיפו useEffect שסורק את ה-DOM לאיתור אלמנטים עם המחלקה rich-media-embed, שולף את הטקסט מתוכם, ומריץ אותו באמצעות הפונקציה window.eval().

הלחץ לספק את הפיצ'ר יצר "דלת אחורית" של ביצוע קוד (Code Execution), שעקפה את כל מעטפת האבטחה החיצונית של השרת.

באיזה הקשר היא מופיעה במערכות ווב

פגיעות זו נפוצה ביישומי Single Page Application (SPA) דינמיים, במיוחד אלו המאפשרים תוכן גולשים עשיר (UGC) ומסתמכים על ארכיטקטורת Session Cookies לצורך אימות מול ממשקי REST API.

מה תוקף יכול להשיג באמצעותה

- **הפצה אקספוננציאלית:** התולעת משתמשת בקורבן כפלטפורמה להדבקת שאר המשתמשים בארגון/מערכת באופן סמוי.
- **Denial of Service (DoS) בלקוח:** נעילת הממשק של הקורבן לחלוטין על ידי הזרקת "מסך מטריקס" קבוע, תוך שימוש במנגנוני התמדה המונעים את הסרתו.
- **ריגול, מעקב ושליטה (C2):** יכולת לקרוא את היסטוריית השיחות של הקורבן, לשתול הודעות בשמו, ולשלוח התראות מודיעיניות בזמן אמת לשיחה של התוקף על כל הדבקה מוצלחת.

סביבת העבודה

טכנולוגיות ומעטפת הגנה עמוקה (Defense in Depth)

הסביבה נבנתה במכוון כ"מבצר" (Fortress) כדי להדגים את עוצמת החולשה הלוגית:

- **צד לקוח (Frontend):** React (JavaScript ES6) עם תקשורת בזמן אמת דרך socket.io-client. סינון קלט מבוצע בעזרת DOMPurify.
- **צד שרת (Backend):** Node.js עם פרימוורק Express. מסד נתונים MongoDB Atlas.
- **מנגנוני אבטחה בשרת:**
  - **CORS מחמיר:** מתיר בקשות אך ורק מהדומיין של הלקוח (http://localhost:5173).
  - **CSP (Content Security Policy) ו-Nonce:** הגדרות נוקשות של script-src 'self' 'nonce-...' ואיסור מסגור תצוגה ('frame-ancestors 'none').
  - **Security Headers (Helmet):** הגדרות X-Frame-Options כ-DENY ו-X-Content-Type-Options כ-nosniff.
  - **ניהול זהויות:** התחברות עם JWT המאוחסן בעוגיית HttpOnly ו-SameSite=Lax.
  - **Anti-CSRF:** מנגנון Token מבוסס State (UUID) נגד זיוף בקשות.
  - **Sanitizer & Rate Limiter:** הגבלת קצב בקשות ל-1 בשנייה, וניקוי HTML קפדני בצד שרת (sanitize-html).

כלים המשמשים לביצוע ההדגמה

- דפדפן רשת סטנדרטי (Chrome).
- כלי מפתחים (DevTools) למעקב אחר התעבורה (Network) ולבחינת מניפולציות ב-DOM.

מימוש והדגמת המתקפה (POC)

תיאור של ה־POC

ה-POC מדגים תולעת רשת (Web Worm) אוטונומית לחלוטין. התולעת נשלחת ארוזה כטקסט בקידוד Base64 בתוך תגית <iframe> המוסווית כסרטון YouTube לגיטימי. ברגע שההודעה מרונדרת במסך של קורבן, התולעת מתעוררת ופועלת כסוכן (Agent) עצמאי בעל מחזור חיים של 5 שלבים:

1. **חילוץ עצמי (Self-Extraction):** קריאת הקוד של עצמה מתוך הזיכרון.
2. **התמדה (Persistence):** נעילת מסך הקורבן ומניעת מחיקתה.
3. **איסוף מודיעין (Reconnaissance):** מיפוי יעדים להדבקה וגניבת הרשאות.
4. **התפשטות (Propagation):** עקיפת מנגנוני הגנה והדבקת היעדים.
5. **שליטה ובקרה (Control - C2 & Command):** פתיחת ערוץ תקשורת חשאי ודיווח חזרה לתוקף.

שלבי הניצול (השלבים של המתקפה) ומעקף ההגנות

התולעת משתלטת על סביבת הקורבן באופן הדרגתי וחכם, תוך עקיפת ההגנות השונות בשרת:

- **שלב 1: הזרקה וחילוץ עצמי (Self-Replication & Bypassing Sanitizers):**

התולעת חומקת ממנגנוני ה-DOMPurify (בלקוח) ו-sanitize-html (בשרת) על ידי הסתתרות כטקסט פנימי בתוך תגית iframe המורשית להכיל סרטוני YouTube. כשהקורבן פותח את השיחה, הלוגיקה הלקויה של האפליקציה קוראת את הטקסט ומפעילה אותו ב-eval(). מיד עם התעוררותה, התולעת סורקת את ה-DOM לאיתור ה-Payload המקורי שלה ומשתמשת בביטויים רגולריים (Regex) כדי לחלץ ולארוז את עצמה מחדש לקראת ההפצה.

- **שלב 2: שיתוק הקורבן והתמדה (Persistence & DoS):**

התולעת מייצרת אלמנט Canvas המכסה את כל המסך ומציג אנימציית "מטריקס" עם ההודעה "SYSTEM COMPROMISED". כדי למנוע מהקורבן להסיר את החסימה באמצעות כלי המפתחים (DevTools), התולעת ממגנת את עצמה באמצעות אובייקט MutationObserver המאזין לשינויים ב-DOM, בשילוב טיימר "כלב שמירה" (setInterval). כל ניסיון מחיקה גורר שחזור מיידי של מסך הנעילה.

- **שלב 3: מיפוי יעדים וגניבת הרשאות (Privilege Escalation):**

התולעת מנצלת את העובדה שהיא רצה ב-Same-Origin כדי "לעוור" את מנגנון ה-CORS. היא משתמשת בפקודת fetch ששואבת אוטומטית את העוגיות המאובטחות (HttpOnly) של הקורבן, ופונה ל-/api/csrf-token כדי לגנוב אסימון CSRF חוקי וטרי. לאחר מכן, היא מושכת את רשימת כל השיחות הפעילות של המשתמש.

- **שלב 4: התפשטות ועקיפת מגבלות קצב (Bypassing Rate Limiter):**

בשרת הוגדר Rate Limiter החוסם בקשות ל-/api/messages בהפרש של פחות משנייה. כדי לעקוף זאת, התולעת משלבת אלגוריתם חזרות (Retry Logic) חכם: היא ממתינה 1.2 שניות לפני כל הדבקה, ואם בכל זאת מתקבלת שגיאת חסימה (HTTP 429), היא ממתינה אוטומטית 2 שניות נוספות ומנסה שוב. כך היא עוברת על כל אנשי הקשר ומדביקה אותם בזה אחר זה.

- **שלב 5: דיווח לתוקף (C2 Exfiltration):**

התולעת מכילה מנגנון דיווח מתוחכם. היא מחפשת האם קיימת שיחה פעילה בין הקורבן לבין מזהה התוקף (attackerId). אם אין שיחה כזו, היא **יוצרת אותה באופן יזום** מאחורי הקלעים (POST /conversations/private). לאחר מכן, היא שולחת לתוקף דיווח בזמן אמת: [WORM REPORT] New victim infected: [Username], וכך מספקת לתוקף תמונת מצב מדויקת של קצב התפשטות המגפה.

Payloads / קוד / בקשות לדוגמה

קוד ההזרקה הראשוני (בתוך ה-iframe כדי לעבור סניטציה):

HTML

```
<iframe class="rich-media-embed"
src="https://www.youtube.com/embed/dQw4w9WgXcQ">

window.eval(window.atob("[BASE64_ENCODED_WORM]"))

</iframe>
```

קטע מתוך קוד התולעת המדגים את לוגיקת החזרות ועקיפת ה-Rate Limiter:

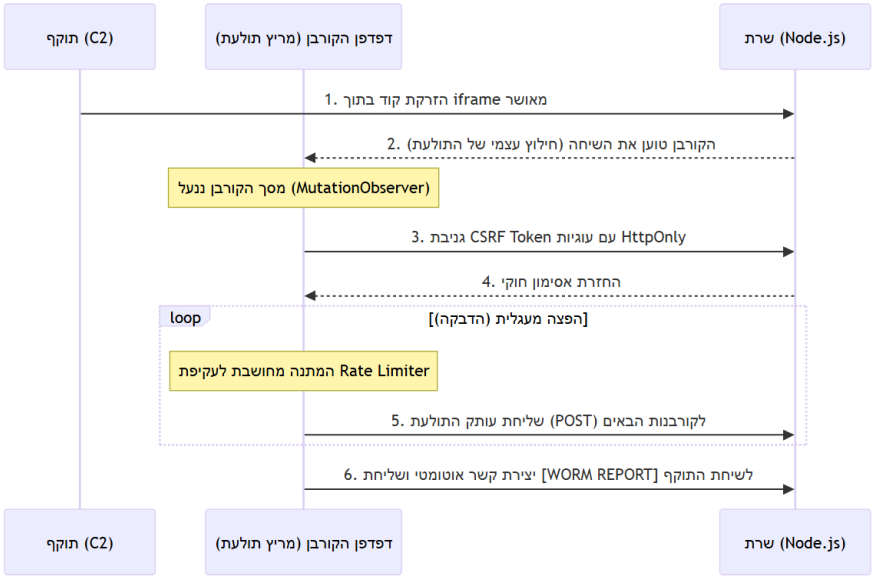
JavaScript

```
async function sendWithRetry(url, options, maxRetries = 3) {

for (let retry = 0; retry < maxRetries; retry++) {

const response = await f(url, options);

if (response.status === 429) { // HTTP 429: Too Many Requests

console.log('Rate limited, waiting 2 seconds...');

await new Promise(r => setTimeout(r, 2000));

continue;

}

return response;

}

}
```

תוצאה בפועל של הניצול

בעת הרצת התרחיש: מיד עם קריאת ההודעה, הקורבן אבד שליטה על הממשק שנגעל לחלוטין. התולעת זיהתה שהמשתמש אינו מוגדר כ-"BOSS" (התוקף), ולכן החלה לבצע רצף פעולות אסינכרוניות ברקע. בתוך שניות ספורות (בכפוף להשהיות ה-Rate Limiter), ההודעה הנגועה הופיעה אצל כל אנשי הקשר של הקורבן. במקביל, נפתחה מאחורי הקלעים שיחה ישירה עם התוקף, והתקבלו בה הודעות מערכת המספקות מידע מודיעיני אודות ההדבקה: [WORM REPORT] New victim infected.

דיאגרמה המתארת את שלבי המתקפה



מנגנוני הגנה ומניעה

מדוע ההגנות הקיימות נכשלו

הפרויקט מוכיח נקודה קריטית באבטחת מידע: **הגנות היקפיות (Perimeter Defenses) אינן מועילות אם קיימת פרצה בלוגיקה הפנימית.**

- **CORS ו-CSRF:** מנגנונים אלו נועדו לעצור אתרים זדוניים *מבחוץ*. ברגע שהתוקף הצליח להריץ קוד *מבפנים* (בזכות ה-eval), הוא הפך למשתמש לגיטימי לכל דבר וה-CORS נוטרל מאחר והבקשה היא Same-Origin.
- **HttpOnly Cookies:** מנגנון זה מנע גניבת קוקיז ישירה (document.cookie), אך לא מנע את השימוש בהם על ידי הדפדפן. בקשות ה-fetch של התולעת נהנו מ"סמכות מובלעת" (Ambient Authority).
- **CSP (Content Security Policy):** ה-CSP מנע טעינת קבצים זדוניים מבחוץ, אך התולעת עשתה שימוש בטכניקת "Living off the Land" – ניצול פונקציות מסוכנות שכבר קיימות באפליקציה החוקית.
- **Sanitizers:** סוננו תגיות אסורות, אך מכיוון שהתוכן הזדוני אוחסן כטקסט פשוט בתוך תגית מותרת (iframe), הסניטציה עברה בהצלחה. הלוגיקה הלקויה בלקוח היא זו שהפכה את הטקסט לקוד.

כיצד ניתן למנוע את החולשה (והיזהרות מפתרון שגוי)

ההגנה חייבת להתמקד בעקירת ה-Code Execution מהשורש:

1. **הסרה מוחלטת של הפונקציה window.eval():** אין להמיר קלט משתמש, מסונן ככל שיהיה, לכדי סקריפט בר-ביצוע.
2. **הקשחת ה-CSP:** וידוא כי ה-CSP חוסם ביצוע של eval על ידי מניעת הדגל 'unsafe-eval'.
3. **מימוש בטוח של מולטימדיה (הימנעות מ-Sandbox Escape):** אם נדרש פיצ'ר " Rich Media", יש לרנדר רכיבי <iframe> המבודדים באמצעות מאפיין sandbox קפדני. חובה להימנע מהשילוב המסוכן של allow-scripts יחד עם allow-same-origin, אשר פותח פתח לבריחה מהארגז (Sandbox Escape) ומאפשר גישה ל-window.parent.

Best Practices רלוונטיים

- **Secure by Design:** דרישות עסקיות (כמו פיצ'ר הווידאו שהוביל לפרצה) חייבות לעבור סקירת אבטחה טרם מימושן. דד-ליין אינו מהווה הצדקה לשימוש בפונקציות מסוכנות.
- **התאמת מנגנונים (Tuning):** מנגנון Rate Limiting חייב להיות מבוסס על "דליי-אסימונים" (Token Bucket) ולמדוד אנומליות התנהגותיות לאורך זמן (למשל, 10 הודעות בדקה), ולא להסתפק במדידת מיקרו-הפרשים בין בקשות סמוכות.

מקורות

מאמרים

- The Danger of eval() in JavaScript Applications.
- Bypassing Rate Limiters using Timing Attacks and Delays.
- PortSwigger: Exploiting CSRF vulnerabilities by chaining with XSS.

תיעוד רשמי

- Mozilla Developer Network (MDN): Never use eval()!
- MDN Web Docs: CORS (Cross-Origin Resource Sharing).
- MDN Web Docs: HttpOnly cookies and Ambient Authority.
- MDN Web Docs: Iframe Sandbox and allow-same-origin vulnerabilities.

OWASP

- OWASP Top 10: A03:2021 – Injection.
- OWASP Cross Site Scripting (XSS) Prevention Cheat Sheet.
- Token Management & OWASP CSRF Prevention.

קישורים רלוונטיים

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/eval
- https://owasp.org/www-community/attacks/xss/

עכשיו יש לך דוח שמשלב בצורה מושלמת את כל המאפיינים הייחודיים של התולעת ושל ההגנות בשרת. מוכן להגשה! האם תרצה שנעבור להכנת קובץ ה-README.md כדי להשלים את ההגשה ב-GitHub?